

ShopperHub

Master Technical Document

Software Requirements · Architecture Review · Workflow Documentation

Version: 2.0 | **Date:** May 2026 | **Status:** Draft — Pre-Production

Repository: perfectprachi-beep/ShopperHub | **Branch:** main

Prepared by: Principal Software Architect | **Classification:** Confidential

Stack: React 18 · Node.js/Express 5 · PostgreSQL · Razorpay · Fast2SMS · Twilio

Part 1 · SRD

Part 2 · Architecture Review

Part 3 · Workflow Docs

68 FR · 21 NFR · 10 Phases

Table of Contents

PART 1 — SOFTWARE REQUIREMENTS DOCUMENT

1.1 Project Overview	4
1.2 Functional Requirements	5
1.3 Non-Functional Requirements	9
1.4 Data Requirements	11
1.5 User Roles & Permissions	13
1.6 API Interface Requirements	14
1.7 Assumptions & Constraints	16
1.8 Glossary	17

PART 2 — ARCHITECTURE & CODE REVIEW

2.1 Architecture Assessment	18
2.2 Code Quality Review	19
2.3 Security Audit	20
2.4 Performance Analysis	21
2.5 Critical Gaps	21
2.6 Review Scorecard	22
2.7 Top 5 Priority Fixes	22

PART 3 — COMPLETE E-COMMERCE WORKFLOW

3.1 System Workflow Overview	23
3.2 Phase-by-Phase Workflow (10 Phases)	24
3.3 Stock Management Decision Tree	30
3.4 Transaction Safety Map	31
3.5 Data Flow Diagrams	32
Executive Summary	33

PART 1

Software Requirements Document

Complete functional and non-functional requirements derived from source code analysis. Covers all modules including product management, inventory, cart, orders, payments, loyalty points, and admin operations.

1.1 Project Overview

Project Name & Purpose

ShopperHub is a full-stack, multi-category fashion and lifestyle e-commerce platform modelled after ShoppersStop. It enables customers to browse products across categories (Men, Women, Kids, Beauty, Luxe, Home), manage carts and wishlists, checkout via Razorpay online payment or Cash on Delivery, track orders, and earn/redeem loyalty points. Administrators manage the full product catalogue, inventory, orders, banners, coupons, CMS content, delivery configuration, and store pickup operations through a dedicated admin panel.

Tech Stack

Layer	Technology	Version	Notes
Frontend	React + Vite	18.3 / 5.4	SPA, lazy-loaded routes, PWA-enabled
State Management	Zustand	4.5	Auth, cart, wishlist, toast stores
Styling	Tailwind CSS	3.4	Utility-first; custom config
Routing	React Router	v6.23	Nested routes, ProtectedRoute guard
Backend	Node.js + Express	22 / 5.0	ESM modules throughout
Database	PostgreSQL	Latest	Raw SQL via pg.Pool; port 5433
Auth	JWT + bcrypt	9.0 / 5.1	Access + refresh tokens; httpOnly cookies
Payments	Razorpay	2.9	Order create + HMAC verification
SMS	Fast2SMS + Twilio	—	India (+91) vs international routing
File Processing	Multer + Sharp	—	Upload + resize images; local /uploads
Security	Helmet + cors + rate-limit	7.1	CSP, CORS, 1000 req/15min global limit
PWA	Vite PWA + Workbox	—	Offline caching for assets & fonts

Architecture Pattern

Layered Monolith — Frontend SPA served separately from backend API. Backend follows strict layering: `Routes → Controllers → Services/Queries → Database`. No microservices; single PostgreSQL instance. Horizontally scalable via stateless JWT auth, but local file storage is a current scaling blocker.

1.2 Functional Requirements

Authentication Module

ID	Description	Input	Output	Business Logic
FR-001	User Registration	full_name, email, phone, password	JWT access token + user object	bcrypt hash (10 rounds); issue access + refresh JWT; set httpOnly cookie
FR-002	User Login	email, password	JWT tokens + user	Verify bcrypt hash; rate-limited 10 req/min via authLimiter
FR-003	Token Refresh	Refresh token (httpOnly cookie)	New access token	Verify refresh JWT secret; issue new short-lived access token
FR-004	Logout	Bearer token	Success message	Clear cookie; note: no server-side revocation store
FR-005	OTP Send	phone number	OTP sent via SMS	Route: +91 → Fast2SMS; other → Twilio; 6-digit OTP
FR-006	OTP Verify	phone, OTP	Verified status	Time-limited OTP validation
FR-007	Admin Login	email, password	JWT with role=admin	Same flow as FR-002; adminGuard enforces role check on all /api/admin/* routes

Product Management Module

ID	Description	Input	Output	Business Logic
FR-008	List Products (storefront)	category, brand, gender, minPrice, maxPrice, minDiscount, sort, page, limit	Paginated product list with total_count	Recursive CTE for subcategory inclusion; only status='active' products shown
FR-009	Product Detail	slug	Product + images + variants + attributes	Three parallel queries; only active products returned
FR-010	Search Products	q (query string)	Ranked product list	PostgreSQL full-text <code>tsvector</code> + <code>ts_rank</code> ; ILIKE fallback on brand/category name
FR-011	Sort Products	sort param	Ordered list	Enum: price_asc, price_desc, newest, discount, random (RANDOM())
FR-012	Admin: Create Product	title, slug, brand_id, category_id, description, gender, base_price, discount_pct, stock, status, is_deal, is_returnable + images	Created product	Images uploaded via Multer; processed by Sharp; stored in /uploads; first image set as primary
FR-013	Admin: Update Product	id + product fields	Updated product	Full field replacement; returns 404 if not found
FR-014	Admin: Soft-Delete Product	id	Success	Sets status='inactive'; product removed from storefront immediately
FR-015	Admin: Bulk Image Fill	— (none)	Count of filled products	Finds active products with no images; assigns placeholder via <code>getProductPlaceholder()</code>

Variant & Inventory Management Module

ID	Description	Input	Output	Business Logic
FR-016	Add Product Variant	product_id, size, color, sku, stock, extra_price	Created variant	Each variant has independent stock; no auto-creation of inventory row

FR-017	Update Variant	variant_id + fields	Updated variant	Full replacement of size, color, sku, stock, extra_price
FR-018	Delete Variant	variant_id	Success	Hard delete; cascades via FK constraints
FR-019	Set Variant Stock (Inventory Module)	variantId, stock_quantity, low_stock_threshold, notes	Inventory record + updated variant	<code>setVariantStock()</code> : upserts inventory row; updates product_variants.stock atomically; creates inventory log
FR-020	Adjust Stock (delta)	inventory_id, quantity_change, action_type, notes	Updated inventory	Adds/subtracts from current stock; logs adjustment; syncs product_variants
FR-021	Bulk Stock Update	updates[] array of {id, stock_quantity}	Updated records	Sequential updates; no transaction wrapper — partial failure leaves inconsistent state
FR-022	Low Stock Alert List	threshold, page, limit	Variants below threshold	Compares inventory.stock_quantity < low_stock_threshold
FR-023	Warehouse CRUD	name, location, manager_name, contact_number	Warehouse record	Soft-delete (is_active=false); inventory items remain orphaned
FR-024	Inventory Dashboard Stats	—	total_products, total_warehouses, in_stock, low_stock, out_of_stock counts	Parallel queries across warehouse + inventory + product_variants tables
FR-025	Inventory Logs	page, limit, filters	Paginated audit log	Types: order_deduction, return_restock, manual_adjustment, variant_stock_set

Cart Management Module

ID	Description	Input	Output	Business Logic
FR-026	View Cart	userId (from JWT)	items[], subtotal, itemCount	JOIN product/variant/image; <code>buildResponse()</code> computes lineTotal server-side
FR-027	Add to Cart	productId or variantId, quantity	Updated cart	If no variantId: find/create default variant (null size/color); stock check before add; UPSERT on conflict
FR-028	Update Cart Quantity	itemId, quantity	Updated cart	quantity must be ≥ 1 ; ownership validated (user_id match)
FR-029	Remove Cart Item	itemId	Updated cart	Hard delete; ownership validated
FR-030	Clear Cart	userId	Empty cart	Auto-triggered inside <code>fulfillOrder()</code> transaction after successful order
FR-031	Guest Cart Merge	guestItems[] array (variantId, quantity)	Merged DB cart	On login: client sends localStorage cart items; server upserts each into DB cart

Order Lifecycle Module

ID	Description	Input	Output	Business Logic
FR-032	Create Razorpay Order	addressId, couponCode, usePoints, deliveryType	razorpayOrderId, amount, keyId, subtotal, discount, shipping, total	Batch stock pre-check (ANY query); coupon validate; points cap 20%; <code>calcShipping()</code> ; Razorpay API call
FR-033	Verify Payment & Fulfill	razorpayOrderId, razorpayPaymentId, razorpaySignature, addressId, couponId, totals, deliveryType	orderId + success message	HMAC-SHA256 verify; calls <code>fulfillOrder()</code> atomic transaction; SMS confirmation
FR-034	Place COD Order	addressId, couponCode, usePoints, deliveryType, storeId	orderId + success message	Same as FR-032 without Razorpay; payment_status='pending'; direct fulfillment

FR-035	Order Fulfillment (atomic)	Full order data + items[]	Confirmed order record	BEGIN → INSERT orders → INSERT order_items → FOR UPDATE lock variants → validate stock ≥ qty → UPDATE stock → sync inventory → award points → mark coupon used → DELETE cart → COMMIT; ROLLBACK on any failure
FR-036	Cancel Order	orderId, userId	Cancelled order	Only status IN ('pending','confirmed'); transaction: set cancelled → restore variant stock → restore inventory → log return_restock → deduct earned points; COMMIT
FR-037	View Order History	userId, page, limit	Paginated orders with items (JSON_AGG)	Grouped by order; items aggregated per order in single query
FR-038	Order Detail	orderId	Order + items + address + store info	Ownership check (user_id = \$2); includes store timing for pickup orders
FR-039	Admin: List All Orders	page, limit, status filter	Paginated orders with customer info	Joined with users; total_count via window function
FR-040	Admin: Update Order Status	orderId, status	Updated order + notification	Status transitions: confirmed → shipped → delivered; fires in-app notification

Payment Integration Module

ID	Description	Input	Output	Business Logic
FR-041	List Payment Methods	—	Active payment methods	Public endpoint; admin-configurable via <code>payment_methods</code> table
FR-042	Razorpay Order Create	amount (₹), currency, receipt	Razorpay order object	<code>Math.round(amount * 100)</code> converts to paise; <code>payment_capture: 1</code> for auto-capture
FR-043	Payment Signature Verify	orderId, paymentId, signature	Boolean valid/invalid	HMAC-SHA256: <code>orderId paymentId</code> signed with RAZORPAY_KEY_SECRET; constant-time comparison
FR-044	Admin: Configure Payment Methods	method, is_active, display config	Updated config	Toggle Razorpay / COD availability for storefront

Coupon & Discount Engine

ID	Description	Input	Output	Business Logic
FR-045	Validate & Apply Coupon	couponCode, subtotal, userId	{valid, discount, couponId, reason}	Chain: is_active → not expired → min_order met → per_user_limit not exceeded → global max_uses not exceeded → compute discount (flat or %)
FR-046	List Active Coupons	—	Public coupon list	Returns is_active + not expired coupons for storefront display
FR-047	Admin: Coupon CRUD	code, discount_type, discount_value, min_order, max_uses, per_user_limit, expires_at	Coupon record	Full CRUD; used_count tracked; unique code constraint
FR-048	Mark Coupon Used	couponId	Incremented used_count	Called inside <code>fulfillOrder()</code> transaction; atomically committed with order

Loyalty Points (First Citizen Points)

ID	Description	Input	Output	Business Logic
FR-049	Earn Points on Order	order total	Points credited to user	1 point per ₹100 (floor); awarded inside <code>fulfillOrder()</code> transaction atomically
FR-050	Redeem Points at Checkout	usePoints flag, current points balance, subtotal	pointsDiscount amount	Cap: MIN(user_points, FLOOR((subtotal - coupon_discount) × 0.20)); 1 point = ₹1
FR-051	Deduct Points on Cancel	orderId, points_earned	Reduced balance	GREATEST(points - earned, 0) inside cancel transaction; prevents negative balance

FR-052	Points Balance	userId	first_citizen_points	Returned in user profile; non-negative guaranteed by DB constraint
--------	----------------	--------	----------------------	--

Storefront / Homepage & Delivery

ID	Description	Input	Output	Business Logic
FR-053	Homepage Sections	—	Banners, featured products, categories, offers	Parallel DB queries; only active records returned
FR-054	Shipping Calculation	deliveryType, subtotal	Shipping amount (₹)	Standard: ₹0 if subtotal ≥ ₹999 else ₹99; Express: ₹199 flat; Store Pickup: ₹0
FR-055	Express Delivery Check	pincode	is_express, delivery_hrs	Lookup in <code>express_delivery_pincodes</code> table; returns availability flag
FR-056	Store Pickup	—	Active stores with address + timing	is_active=true stores only; returns timing for customer display
FR-057	Store Pickup PIN	orderId (store_pickup orders)	4-digit PIN	Generated at order creation: <code>Math.floor(1000 + Math.random() * 9000)</code>
FR-058	CMS Pages	slug	Page content	Admin-managed; rendered at /pages/:slug on frontend

Admin Dashboard Module

ID	Description	Input	Output	Business Logic
FR-059	Dashboard Stats	—	totalOrders, revenue, totalUsers, totalProducts, recentOrders[], salesByDay[]	6 parallel DB queries; 30-day rolling sales chart; only paid orders in revenue
FR-060	Banner CRUD	image (upload or URL), link, position	Banner record	Supports file upload OR URL; is_active toggleable
FR-061	Offer Management	title, discount_pct, target_type, target_id, expires_at	Offer record	Polymorphic target (product/category); shown on product detail page
FR-062	User Management	userId, is_active	Updated user	Cannot modify/delete admin accounts (protected by query WHERE role != 'admin')
FR-063	Notification Send	userId, message	Notification record	In-app; triggered on order status changes; fire-and-forget outside transaction
FR-064	Pickup Status Update	orderId, status	Updated pickup_status	FSM: pending → ready → collected → expired

1.3 Non-Functional Requirements

ID	Category	Requirement	Acceptance Criteria
NFR-001	Security	Password hashing with bcrypt	Min 10 salt rounds; no plaintext passwords in DB or logs
NFR-002	Security	JWT token lifecycle management	Access tokens short-lived; refresh tokens in httpOnly, Secure, SameSite cookies
NFR-003	Security	Content Security Policy	Helmet CSP allowlist: only Razorpay, Google Fonts, and self; no wildcard sources
NFR-004	Security	Auth rate limiting	Max 10 requests/min/IP on all /api/auth/* routes
NFR-005	Security	Global rate limiting	Max 1000 requests/15min/IP across all API routes
NFR-006	Security	Payment signature verification	Every Razorpay payment verified server-side via HMAC-SHA256 before order fulfillment
NFR-007	Security	Admin role enforcement	All /api/admin/* routes require both authGuard (JWT) + adminGuard (role='admin')
NFR-008	Security	SQL injection prevention	100% parameterized queries via pg; no string interpolation in SQL
NFR-009	Performance	Response compression	All API responses gzip-compressed via compression middleware
NFR-010	Performance	DB connection pooling	Single pg.Pool shared across all requests; no per-request connection overhead
NFR-011	Performance	Batch stock pre-check	Stock validation uses single ANY(\$1) query — not N+1 — before order creation
NFR-012	Performance	Image optimization	All uploads processed by Sharp before storage; appropriate dimensions and quality
NFR-013	Reliability	DB connection resilience	connectDB() retries up to 5 times with exponential backoff (2s, 4s, 6s, 8s, 10s)
NFR-014	Reliability	Atomic order fulfillment	fulfillOrder() uses BEGIN/FOR UPDATE/COMMIT; full ROLLBACK on any failure
NFR-015	Reliability	Process crash prevention	unhandledRejection + uncaughtException handlers log and exit cleanly (code 1)
NFR-016	Scalability	Stateless authentication	JWT-based; no server-side session store; horizontal scaling possible
NFR-017	Scalability	12-factor config	All secrets and URLs via .env; no hardcoded values in source (post-remediation)
NFR-018	Scalability	Health & readiness endpoints	/api/health (liveness) and /api/ready (DB ping) for orchestration probes
NFR-019	Maintainability	Layered architecture	Business logic in controllers/services only; no logic in route handlers or query files
NFR-020	Availability	PWA offline support	Workbox caches all static assets + Google Fonts; app shell available offline
NFR-021	Data Integrity	Non-negative stock guarantee	GREATEST(stock - qty, 0) on deduction; FOR UPDATE row lock prevents race conditions

1.4 Data Requirements

Core Entities

Entity / Table	Key Fields	Relationships	Constraints
users	id, full_name, email, phone, password_hash, role, first_citizen_points, is_active	→ orders, addresses, cart_items, wishlists, reviews	email UNIQUE; role ENUM(user/admin); points ≥ 0
products	id, title, slug, brand_id, category_id, sub_category_id, base_price, discount_pct, gender, stock, status, is_deal, is_returnable	→ product_variants, product_images, product_attributes; ← brands, categories	slug UNIQUE; status ENUM(active/inactive); stock fallback when no variants
product_variants	id, product_id, size, color, sku, stock, extra_price	→ cart_items, order_items, inventory	stock ≥ 0; default variant: size=NULL, color=NULL
product_images	id, product_id, url, is_primary, sort_order	← products	One primary per product enforced by app logic
product_attributes	id, product_id, label, value, sort_order, section	← products	section ENUM(highlights/specifications)
categories	id, name, slug, parent_id	self-referential parent/child	slug UNIQUE; recursive CTE used in product queries
brands	id, name, slug, logo_url	→ products	slug UNIQUE
cart_items	id, user_id, product_id, variant_id, quantity	← users, product_variants	UNIQUE(user_id, variant_id) for upsert logic
orders	id, user_id, address_id, coupon_id, subtotal, discount, shipping, total, points_earned, payment_method, payment_status, status, delivery_type, delivery_method, store_id, pickup_status, pickup_pin, expected_by, razorpay_order_id	← users, addresses; → order_items; ← coupons, stores	status ENUM; payment_status ENUM; delivery_type ENUM
order_items	id, order_id, product_id, variant_id, quantity, unit_price	← orders, products, product_variants	unit_price snapshot at purchase time

addresses	id, user_id, label, line1, line2, city, state, pincode, full_name, phone, is_deleted	← users	Soft-deleted; is_deleted=true never shown
coupons	id, code, discount_type, discount_value, min_order, max_uses, used_count, per_user_limit, expires_at, is_active	→ orders	code UNIQUE; used_count increments atomically
inventory	id, variant_id, warehouse_id, stock_quantity, low_stock_threshold	← product_variants, warehouses	NOT auto-created on variant add; manually registered via Inventory module
inventory_logs	id, inventory_id, action_type, quantity, notes, admin_id, created_at	← inventory	action_type ENUM(order_deduction/return_restock/manual_adjustment/variant_stock_set)
warehouses	id, name, location, manager_name, contact_number, is_active	→ inventory	Soft-deleted via is_active=false
stores	id, name, city, state, address, pincode, timing, is_active	→ orders (store_id)	Pickup locations only; separate from warehouses
banners	id, image_url, link, position, is_active	standalone	position for display ordering
offers	id, title, discount_pct, target_type, target_id, expires_at, is_active	polymorphic target	target_type ENUM(product/category)
notifications	id, user_id, message, read_at, created_at	← users	read_at=NULL means unread
reviews	id, user_id, product_id, rating, comment, created_at	← users, products	hasPurchasedProduct() guard; rating 1-5
pages	id, title, slug, content, is_active	standalone	slug UNIQUE; CMS content
payment_methods	id, method, is_active, display_config	standalone	Admin-configured; controls storefront payment options
express_delivery_pincodes	pincode, city, is_express, delivery_hrs	standalone	PK = pincode (6 digits)

Stock Source-of-Truth Rules (COALESCE Logic)

Scenario	Stock Source	Query Used
Product has variants	SUM(product_variants.stock)	COALESCE((SELECT SUM(pv.stock) FROM product_variants pv WHERE pv.product_id = p.id), p.stock)

Product has no variants	products.stock (fallback)	Same COALESCE — returns p.stock when subquery is NULL
Order fulfillment deduction	product_variants.stock (primary) + inventory.stock_quantity (synced)	FOR UPDATE lock → UPDATE product_variants → UPDATE inventory
Inventory module view	inventory.stock_quantity (display only)	LEFT JOIN — variant shown even without inventory record

1.5 User Roles & Permissions

Role	Module	Read	Write	Delete	Notes
Guest	Products / Search / Categories	✓	✗	✗	Browse-only
Guest	Cart	✓ (local)	✓ (localStorage)	✓ (local)	No DB cart; merged on login
Guest	Orders / Wishlist / Reviews	✗	✗	✗	Must register
Customer	Products / Search / Banners	✓	✗	✗	Storefront only
Customer	Cart (DB)	✓	✓	✓ own items	Authenticated; variant stock checked
Customer	Orders	✓ own	✓ (place)	✓ cancel (pending/confirmed)	Cannot view others' orders
Customer	Addresses	✓ own	✓	✓ (soft)	Soft-delete only
Customer	Reviews	✓	✓ (purchased)	✗	hasPurchasedProduct() gate
Customer	Loyalty Points	✓ own	System-managed	✗	Auto earn/deduct by system
Admin	All Admin Routes	✓	✓	✓	role='admin' enforced by adminGuard
Admin	User Management	✓	✓ (non-admin)	✓ (non-admin)	Cannot modify other admin accounts
Admin	Inventory / Warehouse	✓	✓	✓ (soft)	Full access; logs all changes
Admin	Order Status	✓	✓ (any status)	✗	No hard delete of orders

1.6 API Interface Requirements

Base: <http://localhost:5000/api> | Envelope: { success, data?, message? }

Auth, Products & Cart

Method	Endpoint	Auth	Description
POST	/auth/register	None	Register user
POST	/auth/login	None	Login (rate-limited)
POST	/auth/refresh	Cookie	Refresh access token
POST	/auth/logout	Bearer	Logout
POST	/auth/send-otp	None	Send OTP via SMS
POST	/auth/verify-otp	None	Verify OTP
GET	/products	None	List with filters/sort/page
GET	/products/:slug	None	Product detail + variants + images + attrs
GET	/products/search?q=	None	Full-text search
GET	/cart	Bearer	View cart
POST	/cart	Bearer	Add item (productId or variantId)
PUT	/cart/:itemId	Bearer	Update quantity
DELETE	/cart/:itemId	Bearer	Remove item
POST	/cart/merge	Bearer	Merge guest cart on login

Payments, Orders & Account

Method	Endpoint	Auth	Description
GET	/payments/methods	None	List active payment methods
POST	/payments/create-order	Bearer	Create Razorpay order + pre-checks
POST	/payments/create-cod-order	Bearer	Place COD order + fulfill
POST	/payments/verify	Bearer	Verify Razorpay payment + fulfill order
GET	/payments/orders/:id	Bearer	Fetch order detail
GET	/orders	Bearer	Order history (paginated)
POST	/orders/:id/cancel	Bearer	Cancel order (pending/confirmed only)
GET	/account/profile	Bearer	User profile + points
GET	/account/addresses	Bearer	List active addresses
POST	/account/addresses	Bearer	Add address
PUT	/account/addresses/:id	Bearer	Update address
DELETE	/account/addresses/:id	Bearer	Soft-delete address
GET	/notifications	Bearer	List user notifications
PUT	/notifications/:id/read	Bearer	Mark notification read

Admin Routes (Bearer + role=admin required)

Method	Endpoint	Description
GET	/admin/dashboard/stats	Dashboard KPIs + 30-day sales chart
GET/POST/PUT/DELETE	/admin/products	Product CRUD (images via Multer)
GET/POST/PUT/DELETE	/admin/products/:id/variants	Variant management
GET/POST/PUT/DELETE	/admin/products/:id/attributes	Product attributes
GET/POST/DELETE	/admin/products/:id/images	Image management
POST	/admin/products/fill-missing-images	Bulk placeholder image assignment
GET/POST/PUT/DELETE	/admin/categories	Category CRUD (hierarchical)
GET/POST/PUT/DELETE	/admin/brands	Brand CRUD (with logo upload)
GET/POST/PUT/DELETE	/admin/banners	Banner CRUD
GET/POST/PUT/DELETE	/admin/coupons	Coupon CRUD
GET/POST/PUT/DELETE	/admin/offers	Offer management
GET	/admin/orders	All orders with filters
PUT	/admin/orders/:id/status	Update order status
PUT	/admin/orders/:id/pickup-status	Update pickup FSM status
GET/PATCH/DELETE	/admin/users	User management
GET/POST/PUT/DELETE	/admin/payment-methods	Payment method config
GET/POST/PATCH/DELETE	/admin/delivery/stores	Store pickup management
GET/POST/PATCH/DELETE	/admin/delivery/pincodes	Express delivery pincodes
GET/POST/PUT/DELETE	/admin/pages	CMS page management

Inventory Routes (Bearer + role=admin required)

Method	Endpoint	Description
GET	/inventory/dashboard/stats	Stock counts by status + warehouse
GET/POST/PUT/DELETE	/inventory/warehouses	Warehouse CRUD
GET/POST/PUT	/inventory/inventory	Inventory record CRUD
POST	/inventory/inventory/bulk-update	Bulk stock quantity update
POST	/inventory/inventory/:id/adjust	Stock delta adjustment with log
PUT	/inventory/variants/:variantId/stock	setVariantStock() — upserts inventory + syncs variant
GET	/inventory/low-stock	Low stock alert list
GET/POST	/inventory/logs	Inventory audit log

1.7 Assumptions & Constraints

ID	Type	Description
A-01	Assumption	Currency is INR exclusively; Razorpay amounts converted to paise ($\times 100$) via <code>Math.round()</code>
A-02	Assumption	Phone numbers with +91 prefix route to Fast2SMS; all others to Twilio
A-03	Assumption	Products with no variants use <code>products.stock</code> as fallback; COALESCE handles both cases
A-04	Assumption	Loyalty points never expire; permanent until redeemed or order cancelled
A-05	Assumption	A review can only be submitted after <code>order.status = 'delivered'</code>
A-06	Assumption	Store pickup and warehouses are separate concepts (<code>stores</code> table $\neq$ <code>warehouses</code> table)
A-07	Assumption	Image uploads are processed by Sharp and stored on local disk <code>/uploads</code> (no cloud storage)
C-01	Constraint	Inventory row is NOT auto-created when a variant is added via the Products tab — admin must manually register it in the Inventory module
C-02	Constraint	Guest cart relies on client-side <code>localStorage</code> only — no server-side stock validation until merge on login
C-03	Constraint	Bulk stock update (<code>bulkUpdateInventoryStock</code>) has no transaction wrapper — partial failure leaves inconsistent state
C-04	Constraint	No refresh token revocation store — logout does not truly invalidate tokens until natural expiry
C-05	Constraint	Local file storage blocks horizontal scaling — S3/CDN migration required for multi-server deployment
C-06	Constraint	No API versioning (<code>/api/v1/</code>) — breaking changes affect all clients immediately
C-07	Constraint	No automated test suite — all testing is manual; no CI/CD pipeline configured
C-08	Constraint	Migration scripts are numbered manually (<code>migrate.js</code> , <code>migrate-phase5.js...</code>) with no migration runner
C-09	Constraint	Razorpay only; no PayPal, UPI QR, or digital wallet support
C-10	Constraint	Store pickup PIN is random (not cryptographically secure); adequate for low-stakes use but not banking-grade

1.8 Glossary

Term	Definition
SKU	Stock Keeping Unit — unique identifier per product variant (stored in product_variants.sku)
COALESCE Stock	SQL pattern used in product queries: COALESCE((SELECT SUM(variant.stock)...), product.stock) — returns variant sum if variants exist, otherwise falls back to product-level stock
First Citizen Points	Loyalty reward program; customers earn 1 point per ₹100 spent; redeemable at checkout (1 pt = ₹1); capped at 20% of post-coupon subtotal per order
setVariantStock()	Server function in inventory.js that upserts an inventory record for a variant AND updates product_variants.stock simultaneously — the canonical way to set stock via the Inventory module
fulfillOrder()	Atomic DB transaction function in orders.js that: inserts order, inserts items, locks variants (FOR UPDATE), validates stock, deducts stock, syncs inventory, awards points, marks coupon used, clears cart — all in one BEGIN/COMMIT block
authGuard	Express middleware that validates JWT Bearer token and attaches req.user; applied to all protected routes
adminGuard	Express middleware that enforces req.user.role === 'admin'; applied after authGuard on all /api/admin/* and /api/inventory/* routes
lineTotal	Cart item subtotal computed as unitPrice × quantity in the DB cart query; returned pre-computed in cart response
paise	Indian sub-unit of currency; 1 INR = 100 paise; Razorpay API requires all amounts in paise (Math.round(amount × 100))
Pickup PIN	4-digit random code generated at store-pickup order creation for collection
Soft Delete	Setting is_deleted=true (addresses) or is_active=false (warehouses) instead of removing the DB row
Default Variant	A product_variants row with size=NULL and color=NULL, auto-created by cart.controller.js when a product without explicit variants is added to cart
buildResponse()	Cart controller helper that computes subtotal and itemCount from raw cart items array; ensures consistent cart response shape
calcShipping()	Shared function in payments.controller.js; determines shipping cost from deliveryType and subtotal; single source of truth for shipping logic

PART 2

Architecture & Code Review

Independent assessment of code quality, security posture, performance characteristics, and architectural integrity. Findings reference actual function names, file paths, and SQL patterns found in the source code.

2.1 Architecture Assessment

Dimension	Finding	Status
Pattern	Layered Monolith — React SPA + Express REST API. Clean separation of concerns.	✅ Appropriate
Layer Separation	Routes → Controllers → Queries/Services strictly followed. No business logic in route handlers.	✅ Clean
Admin Module	admin.routes.js is 566 lines with inline route handlers — no dedicated admin controllers. Untestable.	⚠️ God File
Coupling	Controllers are loosely coupled via function imports. DB pool is a shared singleton — acceptable.	✅ Acceptable
Reusability	calcShipping(), buildResponse(), getProductPlaceholder() are good shared helpers. More extraction needed.	⚠️ Partial
Config	All secrets via .env — 12-factor compliant. Health + readiness endpoints present.	✅ Good
Frontend Architecture	Zustand stores per concern (auth, cart, wishlist, UI). Lazy-loaded routes with Suspense. PWA-ready.	✅ Well structured

2.2 Code Quality Review

Area	Observation	Status
Naming Conventions	Consistent camelCase throughout; descriptive function names (fulfillOrder, setVariantStock, buildResponse).	✓ Good
DRY Violations	Stock pre-check loop was duplicated in createOrder + createCodOrder (fixed). calcShipping defined once — good.	⚠ Mostly fixed
Function Complexity	CheckoutPage.jsx ~889 lines; fulfillOrder() ~100 lines but justified as a transaction unit.	⚠ CheckoutPage needs splitting
Dead Code	Old createOrder() / addOrderItems() / deductStock() / awardPoints() / markCouponUsed() functions in orders.js are superseded by fulfillOrder() but still exported. Unused.	✗ Remove dead exports
Comments	Meaningful inline comments on complex logic (COALESCE, recursive CTE, FOR UPDATE). Not over-commented.	✓ Good
Console Logs	authGuard.js debug logs removed. Some [createOrder] logs remain — acceptable for dev, gate behind DEBUG env var for prod.	⚠ Minor
Joi Usage	Joi installed but not used anywhere — no request body validation layer exists.	✗ Critical gap
Magic Strings	'standard', 'express', 'store_pickup' delivery type strings used in 5+ places without constants.	⚠ Extract constants

2.3 Security Audit

Check	Finding	Status
Password Hashing	bcrypt with default rounds. No plaintext anywhere.	✓ Secure
JWT Implementation	Short-lived access tokens; refresh in httpOnly cookie. No revocation store.	⚠ Logout gap
SQL Injection	100% parameterized queries. No string interpolation in SQL. Safe.	✓ Secure
Admin Authorization	authGuard + adminGuard applied at router level (router.use(authGuard, adminGuard)). All admin routes covered.	✓ Secure
Payment Security	HMAC-SHA256 signature verification server-side before any order fulfillment. Correct.	✓ Secure
Exposed Secrets (Historical)	Twilio SID + Fast2SMS key were in SMS-NOTIFICATION-GUIDE.md (commit a29ee3f). Scrubbed from history. CREDENTIALS MUST BE ROTATED.	✗ Rotate immediately
CSP Headers	Helmet CSP configured with explicit allowlist — Razorpay + Google Fonts only.	✓ Secure
CORS	Allows CLIENT_URL + any localhost port in dev. Strict in production.	✓ Acceptable
Input Validation	No Joi schemas on any endpoint. Only ad-hoc checks (if !addressId). XSS risk from unvalidated string inputs.	✗ Add validation
Address Ownership	Validated via array scan (addresses.find()) not DB query — inefficient but functionally correct.	⚠ Use DB WHERE user_id=\$1 AND id=\$2
Rate Limiting	Auth: 10/min; Global: 1000/15min. In-memory store — resets on restart; Redis recommended for prod.	⚠ Use Redis for prod

2.4 Performance Analysis

Issue	Location	Impact	Status
N+1 Stock Pre-check (fixed)	payments.controller.js	Was: 1 query per cart item. Now: single ANY(\$1) batch query.	✅ Fixed
Stock loop in fulfillOrder()	orders.js fulfillOrder()	Still loops per item for FOR UPDATE lock — intentional (cannot batch row locks)	✅ Correct
RANDOM() sort	products.js getProducts()	ORDER BY RANDOM() is O(n log n) full table scan — disables index use	⚠️ Cache or precompute
No query indexes documented	All migration files	Missing indexes on: orders.user_id, cart_items.user_id, products.slug, product_variants.product_id	❌ Add indexes
Razorpay singleton	payments.service.js getRazorpay()	New Razorpay instance created per call. Should be module-level singleton.	⚠️ Minor overhead
No caching	Public endpoints	Categories, banners, homepage data queried from DB on every request — cacheable.	⚠️ Add Redis cache
Parallel queries	getProductBySlug(), adminGetDashboardStats()	Promise.all() used correctly for independent queries.	✅ Good

2.5 Critical Gaps Found

#	Gap	Location	Risk	Recommendation
1	Inventory row NOT auto-created on variant add	admin.routes.js POST /products/id/variants	HIGH	Add createInventory() call after addVariant() or add a DB trigger on product_variants INSERT
2	Guest cart has no server-side stock check	localStorage (client only)	HIGH	Cart merge endpoint (POST /cart/merge) should validate stock per item before upserting
3	Bulk stock update has no transaction	inventory.js bulkUpdateInventoryStock()	HIGH	Wrap in BEGIN/COMMIT; partial failure currently leaves inconsistent state
4	No Joi validation on any request body	All controllers	HIGH	Add Joi schemas — library already installed; implement on checkout, auth, and admin routes first
5	Dead code in orders.js	orders.js lines 4-50	MED	Remove createOrder(), addOrderItems(), deductStock(), awardPoints(), markCouponUsed() — all superseded by fulfillOrder()
6	Zero test coverage	Entire codebase	HIGH	Start with: verifySignature(), validateCoupon(), calcShipping(), fulfillOrder() integration test
7	No missing index definitions	Migration files	MED	Add: CREATE INDEX ON orders(user_id); CREATE INDEX ON cart_items(user_id); CREATE UNIQUE INDEX ON products(slug);

2.6 Review Scorecard

Area	Score (1-10)	Status	Priority
Architecture	7.5 / 10	✔ Good	LOW
Code Quality	6.5 / 10	⚠ Acceptable	MEDIUM
Security	6.0 / 10	⚠ Needs work	HIGH
Performance	6.0 / 10	⚠ Missing indexes/cache	MEDIUM
Data Integrity	7.0 / 10	✔ Good (transaction fixed)	MEDIUM
Error Handling	7.0 / 10	✔ Central handler + process guards	LOW
Testing	1.0 / 10	✖ Zero tests	CRITICAL
Overall	5.9 / 10	Pre-production MVP	Not production-ready

2.7 Top 5 Priority Fixes Before Production

#	Issue	Fix	Effort
1	● Rotate exposed credentials	Regenerate Twilio SID/auth token and Fast2SMS API key on their dashboards immediately. Update .env. The old credentials were in git commit a29ee3f and may have been scraped.	30 min
2	● Auto-create inventory on variant add	In admin.routes.js POST /products/:id/variants, after addVariant(), call createInventory({ variant_id: variant.id, stock_quantity: variant.stock }). This closes the gap between the Products tab and Inventory module.	1 hour
3	● Add Joi validation to all endpoints	Joi is installed. Add schemas for: register, login, createOrder, createCodOrder, addToCart, updateQty. Prevents invalid data reaching DB and closes XSS surface.	1 day
4	● Wrap bulk stock update in transaction	In inventory.js bulkUpdateInventoryStock(), wrap all updates in a single client.query('BEGIN') / COMMIT block. If any update fails, ROLLBACK to maintain consistency.	2 hours
5	● Add critical DB indexes	Run migration: CREATE INDEX ON orders(user_id); CREATE INDEX ON cart_items(user_id); CREATE INDEX ON product_variants(product_id); CREATE UNIQUE INDEX ON products(slug). Without these, queries degrade as data grows.	1 hour

PART 3

Complete E-Commerce Workflow Documentation

End-to-end workflow covering all 10 operational phases for Admin, Customer, and Guest actors. Includes phase triggers, API calls, DB tables affected, business rules, and outcomes.

3.1 System Workflow Overview

Admin Actor

The admin accesses the platform at `/admin` and logs in using credentials verified against the users table (role='admin'). Once authenticated, the admin can manage the full product catalogue — creating products, adding size/color variants with stock levels, uploading images, and optionally registering those variants in the Inventory module (linked to a physical warehouse). The admin configures homepage banners, promotional offers, discount coupons, delivery options (express pincodes, store pickup locations), and CMS pages. When orders arrive, the admin reviews them on the Orders dashboard, updates status through the lifecycle (confirmed → shipped → delivered), and monitors inventory health via the Inventory Dashboard with low-stock alerts and audit logs.

Logged-in Customer Actor

A registered customer authenticates via email/password or OTP. On login, any items in their localStorage guest cart are automatically merged into their DB cart. They browse products using category navigation, filters (price, brand, gender, discount), full-text search, and sorting. On the product detail page they select a size/color variant, check stock availability, and add to their DB cart. At checkout, they select a saved address (or add a new one), choose a delivery option (Standard, Express subject to pincode availability, or Store Pickup), optionally apply a coupon code, optionally redeem First Citizen loyalty points (up to 20% of subtotal), and proceed to pay via Razorpay (online) or COD. After payment, the order is atomically fulfilled, their cart is cleared, points are awarded, and an SMS confirmation is sent. They can track orders, view history, cancel eligible orders, and leave reviews on delivered products.

Guest Customer Actor

A guest can browse the full product catalogue, view product details including pricing and availability, and add items to a local cart stored in their browser's localStorage. The app reads variant stock from the API for display purposes. No server-side stock reservation happens for guests. When the guest registers or logs in, their localStorage cart is sent to the POST `/cart/merge` endpoint, which upserts each item into the authenticated DB cart. From that point they follow the customer checkout flow.

3.2 Phase-by-Phase Workflow

PHASE 1 — Product Creation (Admin)

Actor:	Admin
Trigger:	Admin clicks "Add Product" in Admin Panel → Products
Steps:	1. Fill product form (title, brand, category, price, discount%, gender, description, status) 2. Upload product images (Multer receives files; Sharp resizes/optimizes; saved to /uploads) 3. Submit → POST /admin/products
API Called:	POST /admin/products (multipart/form-data with images)
DB Tables Affected:	products (INSERT), product_images (INSERT per file, first=primary)
Business Rules:	slug must be unique; status defaults to 'active'; is_deal and is_returnable default to false/true
Success Outcome:	Product visible in admin list; if status='active' immediately visible on storefront
Failure Outcome:	Duplicate slug → DB unique constraint error → 500 (should be caught + 400)

PHASE 2 — Variant Addition & Stock Setup

Actor:	Admin
Trigger:	Admin opens product → Variants tab → "Add Variant"
Steps:	1. Enter size, color, SKU, stock quantity, extra_price 2. POST /admin/products/:id/variants 3. Variant appears in variant list with stock count ⚠ NOTE: No inventory record is auto-created at this step
API Called:	POST /admin/products/:id/variants
DB Tables Affected:	product_variants (INSERT)
Business Rules:	stock ≥ 0; extra_price added to base_price for final variant price; variant stock overrides products.stock in COALESCE queries
Success Outcome:	Variant selectable on product detail page; stock shown to customers
Failure Outcome:	Variant without inventory record — Inventory module shows it but cannot track warehouse location until Step 3

PHASE 3 — Inventory Module Registration

Actor:	Admin
Trigger:	Admin navigates to Inventory module → assigns variant to warehouse
Steps:	1. Create warehouse if needed: POST /inventory/warehouses 2. Register variant: POST /inventory/inventory {variant_id, warehouse_id, stock_quantity} - OR use the faster: PUT /inventory/variants/:variantId/stock 3. setVariantStock() upserts inventory row + updates product_variants.stock + creates inventory log
API Called:	POST /inventory/warehouses; PUT /inventory/variants/:variantId/stock
DB Tables Affected:	warehouses (INSERT), inventory (UPSERT), product_variants (UPDATE stock), inventory_logs (INSERT with action='variant_stock_set')
Business Rules:	setVariantStock() is the authoritative function — it keeps both tables in sync; inventory.stock_quantity mirrors product_variants.stock after this call
Success Outcome:	Variant appears in Inventory Dashboard; low-stock alerts now active; warehouse association visible
Failure Outcome:	If skipped: variant tracked only in product_variants; inventory module shows it with no warehouse; low-stock alerts inactive for this variant

PHASE 4 — Customer Browses Storefront

Actor:	Guest or Customer
Trigger:	User opens app at <code>http://localhost:5173</code>
Steps:	- Homepage loads → parallel API calls: banners, featured products, categories, offers - User navigates category (e.g., /women) → <code>GET /products?gender=women&category=women</code> - Recursive CTE resolves parent category → all subcategory products included - User applies filters (price, brand, discount) → query params updated → new API call - User clicks product → <code>GET /products/:slug</code> → images + variants + attributes loaded - Stock shown per variant via COALESCE logic
API Called:	<code>GET /home</code> ; <code>GET /products</code> ; <code>GET /products/:slug</code> ; <code>GET /products/search</code>
DB Tables Affected:	Read-only: products, product_variants, product_images, product_attributes, brands, categories, banners, offers
Business Rules:	Only status='active' products shown; COALESCE stock: <code>SUM(variant.stock)</code> if variants exist, else <code>products.stock</code> ; offers displayed if not expired
Success Outcome:	Product catalogue rendered with live stock and pricing
Failure Outcome:	DB unavailable → <code>/api/ready</code> returns 503; frontend shows error state

PHASE 5 — Add to Cart (Logged-in + Guest)

Actor:	Customer (DB cart) or Guest (localStorage)
Trigger:	User clicks "Add to Bag" on product detail page after selecting variant
Steps (Customer):	<code>POST /cart {variantId, quantity}</code> - If no variantId: server finds/creates default variant (<code>size=NULL, color=NULL</code>) - Server checks <code>product_variants.stock > 0</code> → 409 if out of stock <code>addOrUpdateItem()</code>: <code>INSERT ... ON CONFLICT (user_id, variant_id) DO UPDATE SET quantity = quantity + \$qty</code> - Returns updated cart with subtotal and itemCount
Steps (Guest):	- Item added to localStorage array (no server call) - Stock display is from previous product detail API call (not real-time) - On login: <code>POST /cart/merge {items:[]}</code> sends localStorage cart to server ⚠️ No stock validation at merge time — gap exists
API Called:	<code>POST /cart</code> ; <code>POST /cart/merge</code> (on login)
DB Tables Affected:	cart_items (UPSERT), product_variants (READ stock check only)
Business Rules:	Stock must be > 0 to add (authenticated users); UPSERT increments quantity if item exists; quantity must be ≥ 1
Success Outcome:	Item in cart; subtotal updated; cart badge in navbar increments
Failure Outcome:	Out of stock → 409; item not found → 404

PHASE 6 — Checkout & Payment (Razorpay + COD)

Actor:	Customer
Trigger:	Customer clicks "Proceed to Checkout" from cart
Steps (Razorpay):	- Step 1 (Address): Select/add delivery address - Step 2 (Delivery): Choose Standard / Express (pincode check) / Store Pickup - Step 3 (Payment): Enter coupon code → validated server-side; toggle Use Points; select Razorpay <code>POST /payments/create-order</code> → batch stock check (ANY query) → coupon validate → points cap 20% → <code>calcShipping()</code> → Razorpay API creates order - Response: razorpayOrderId + amount + keyId sent to frontend - Razorpay modal opens → customer pays - On success: <code>POST /payments/verify {razorpayOrderId, paymentId, signature, ...totals}</code> - Server: <code>verifySignature()</code> → HMAC match → <code>fulfillOrder()</code> → redirect to /order-success
Steps (COD):	1–3 same as above <code>POST /payments/create-cod-order</code> → same pre-checks → direct <code>fulfillOrder()</code> call <code>payment_status='pending'</code> (paid only on delivery)
API Called:	<code>GET /stores</code> ; <code>GET /account/addresses</code> ; <code>POST /payments/create-order</code> ; <code>POST /payments/verify</code> ; <code>POST /payments/create-cod-order</code>

DB Tables Affected:	orders (INSERT), order_items (INSERT), product_variants (UPDATE stock), inventory (UPDATE), cart_items (DELETE), users (UPDATE points), coupons (UPDATE used_count)
Business Rules:	Shipping: Standard $\geq ₹999$ free else ₹99; Express ₹199; Pickup ₹0. Points: $\text{MIN}(\text{balance}, \text{FLOOR}((\text{subtotal} - \text{discount}) \times 0.20))$. Coupon: validated against active/expired/min_order/per_user/global limits.
Success Outcome:	Order created; cart cleared; SMS sent; in-app notification fired; redirect to order success page
Failure Outcome:	Out of stock $\rightarrow$ 409; invalid signature $\rightarrow$ 400; Razorpay error $\rightarrow$ 502 with message; coupon invalid $\rightarrow$ 400

PHASE 7 — Order Fulfillment & Stock Deduction (atomic)

Actor:	System (fulfillOrder() transaction)
Trigger:	Called from verifyPayment() or createCodOrder() after all pre-checks pass
Steps:	- client.query('BEGIN') - INSERT INTO orders (...) RETURNING * $\rightarrow$ order record created - INSERT INTO order_items (batch values template) $\rightarrow$ all items inserted - FOR EACH item with variantId: - SELECT stock FROM product_variants WHERE id=\$1 FOR UPDATE (row lock) - IF stock < quantity $\rightarrow$ throw 409 error $\rightarrow$ ROLLBACK - UPDATE product_variants SET stock = stock - \$qty WHERE id=\$variantId - UPDATE inventory SET stock_quantity = GREATEST(stock_quantity - \$qty, 0) WHERE variant_id=\$variantId - INSERT INTO inventory_logs (action='order_deduction', qty=-qty) - IF pointsEarned > 0: UPDATE users SET first_citizen_points = first_citizen_points + \$pts - IF couponId: UPDATE coupons SET used_count = used_count + 1 - DELETE FROM cart_items WHERE user_id=\$userId - client.query('COMMIT') (Outside transaction): createNotification() fire-and-forget
DB Tables Affected:	orders, order_items, product_variants, inventory, inventory_logs, users, coupons, cart_items, notifications
Business Rules:	FOR UPDATE prevents race condition on last item. GREATEST prevents negative stock. pointsEarned = FLOOR(total/100). All steps atomic — any failure rolls back completely.
Success Outcome:	Order confirmed; stock decremented in both product_variants and inventory; points awarded; coupon counted; cart empty
Failure Outcome:	Any error $\rightarrow$ ROLLBACK $\rightarrow$ no partial state; customer sees error message; Razorpay payment captured but order not created $\rightarrow$ requires manual reconciliation (gap)

PHASE 8 — Order Cancellation & Stock Restoration

Actor:	Customer or Admin
Trigger:	Customer clicks "Cancel Order" on order detail page; or Admin changes order status
Steps:	- POST /orders/:id/cancel (customer) or PUT /admin/orders/:id/status {status:'cancelled'} - client.query('BEGIN') - UPDATE orders SET status='cancelled' WHERE id=\$1 AND user_id=\$2 AND status IN ('pending','confirmed') RETURNING * - IF no rows returned $\rightarrow$ ROLLBACK (order not in cancellable state) - SELECT variant_id, quantity FROM order_items WHERE order_id=\$1 - FOR EACH item: - UPDATE product_variants SET stock = stock + \$qty WHERE id=\$variantId - UPDATE inventory SET stock_quantity = stock_quantity + \$qty WHERE variant_id=\$variantId RETURNING id - INSERT INTO inventory_logs (action='return_restock', qty=+qty, notes='Order #X cancelled') - IF points_earned > 0: UPDATE users SET first_citizen_points = GREATEST(pts - earned, 0) - client.query('COMMIT')
API Called:	POST /orders/:id/cancel
DB Tables Affected:	orders, product_variants, inventory, inventory_logs, users
Business Rules:	Only pending/confirmed orders cancellable. Stock fully restored. Points from this order deducted (GREATEST prevents negative). Coupon used_count NOT decremented (gap).
Success Outcome:	Order cancelled; stock restored; points adjusted; customer can re-order
Failure Outcome:	Order already shipped/delivered $\rightarrow$ null returned $\rightarrow$ 404 "Order not found or cannot be cancelled"

PHASE 9 — Storefront Visibility & Stock Filters

Actor:	System (query layer)
Trigger:	Any product listing or detail API call
Steps:	1. getProducts() builds WHERE p.status = 'active' as base condition 2. If category filter: recursive CTE resolves all child category IDs 3. Stock computed: COALESCE(SELECT SUM(pv.stock) FROM product_variants pv WHERE pv.product_id = p.id), p.stock) 4. Returns stock alongside each product — frontend uses this to show "In Stock" / "Out of Stock" 5. Variant-level stock shown on product detail page (SELECT * FROM product_variants WHERE product_id=\$1) 6. Admin soft-delete sets status='inactive' → product immediately disappears from all storefront queries
API Called:	GET /products; GET /products/:slug
DB Tables Affected:	Read-only: products, product_variants, categories (recursive CTE)
Business Rules:	COALESCE: if variants exist → SUM(variant.stock); else → products.stock. Frontend filters: stock=0 → "Out of Stock" badge, add-to-cart disabled. Categories: browsing /category/men shows all men + subcategory products via recursive CTE.
Success Outcome:	Accurate real-time stock visibility; category hierarchy browsing works correctly
Failure Outcome:	If inventory row not created but variant exists: stock from product_variants — functionally correct, inventory module just has no warehouse mapping

PHASE 10 — Loyalty Points & Coupon Lifecycle

Actor:	System + Customer
Trigger:	Customer places order / cancels order / applies coupon at checkout
Points Earn:	pointsEarned = FLOOR(total / 100) → awarded inside fulfillOrder() transaction atomically
Points Redeem:	1. Customer toggles "Use Points" at checkout 2. Server fetches user.first_citizen_points 3. maxDiscount = FLOOR((subtotal - couponDiscount) × 0.20) 4. pointsDiscount = MIN(userPoints, maxDiscount) 5. Deducted from order total; separate deduction via UPDATE users after verify (gap: not inside fulfillOrder transaction)
Points on Cancel:	Earned points reversed: UPDATE users SET first_citizen_points = GREATEST(pts - earned, 0) inside cancel transaction
Coupon Flow:	1. Customer enters code → POST /coupons/validate (or inline in create-order) 2. validateCoupon(): active? → not expired? → min_order met? → per_user_limit? → global max_uses? → compute discount 3. Coupon ID stored in order; used_count incremented inside fulfillOrder() transaction ⚠ On cancel: used_count NOT decremented — gap
DB Tables Affected:	users (points UPDATE), coupons (used_count UPDATE), orders (discount, coupon_id)
Business Rules:	Points cap: 20% of post-coupon subtotal. 1 point = ₹1. No expiry on points. Coupon: flat or percentage discount; per-user and global usage limits enforced separately.
Success Outcome:	Points correctly earned, capped, and redeemed. Coupon validated and usage tracked.
Failure Outcome:	Coupon expired/overused → 400 with reason message. Points balance never goes negative (GREATEST guard).

3.3 Stock Management Decision Tree

STOCK SOURCE OF TRUTH DECISION TREE

```
Q1: Does the product have any rows in product_variants?
|
└─ NO → Stock = products.stock (fallback field)
      Shown as: COALESCE(NULL, products.stock) = products.stock
      Edit via: PUT /admin/products/:id {stock: N}
|
└─ YES → Stock = SUM(product_variants.stock) for all variants
      Shown as: COALESCE(SUM(pv.stock), products.stock)
      products.stock is IGNORED when variants exist

Q2: Is this variant registered in the Inventory module?
|
└─ NO → Stock tracked in product_variants only
      No warehouse mapping; no low-stock alert
      Updated via: PUT /admin/products/variants/:id
|
└─ YES → Stock synced across BOTH tables:
      • product_variants.stock (source of truth for orders)
      • inventory.stock_quantity (mirrors; used for dashboard)
      Updated via: PUT /inventory/variants/:variantId/stock
                  → setVariantStock() keeps both in sync
```

STOCK STATE TABLE

State	Storefront	Inventory Module
No variants, products.stock = 0	Out of Stock	Not tracked
No variants, products.stock > 0	In Stock	Not tracked
Variants exist, all stock = 0	Out of Stock	Depends on reg.
Variants exist, some stock > 0	In Stock	Depends on reg.
Variant registered, inv. row exists	In Stock	Tracked + alerts
Order placed → stock decremented	Real-time	Synced + logged
Order cancelled → stock restored	Real-time	Synced + logged

3.4 Transaction Safety Map

Operation	Transaction?	Row Lock?	Tables Modified	Rollback Condition
Order Placement (fulfillOrder)	✓ BEGIN/COMMIT	✓ FOR UPDATE on variants	orders, order_items, product_variants, inventory, inventory_logs, users, coupons, cart_items	stock < quantity ordered; any DB error
Order Cancellation (cancelOrder)	✓ BEGIN/COMMIT	✗ No row lock	orders, product_variants, inventory, inventory_logs, users	Order not in pending/confirmed status
Bulk Stock Update	✗ No transaction	✗ No row lock	inventory, product_variants, inventory_logs	None — partial failure possible
setVariantStock()	⚠ Single client block	✗ No explicit lock	product_variants, inventory (UPSERT), inventory_logs	Variant not found
adjustStock()	⚠ Single client block	✗ No explicit lock	inventory, product_variants, inventory_logs	Inventory item not found
Cart Add/Update	✗ Single query	✗ No lock	cart_items	Stock check is pre-flight only; no atomic reserve
Coupon Validation + Apply	✓ Inside fulfillOrder	✗ No lock on coupon row	coupons (used_count)	TOCTOU risk under high concurrency on last use
Points Redemption Deduction	✗ Outside fulfillOrder	✗ No lock	users	None — race condition possible under concurrency

3.5 Data Flow Diagrams

Product Creation Flow

```
[Admin UI: /admin/products/new]
| POST /admin/products (multipart/form-data)
▼
[admin.routes.js: POST /products]
| authGuard → adminGuard → uploadImages (Multer)
▼
[createProduct(req.body)] → [DB: INSERT INTO products]
|                               |
| req.files.forEach → addImage(productId, url) |
|                               |
▼                               ▼
[DB: INSERT INTO product_images]      [products row RETURNING *]
|                                     |
▼                                     ▼
[Response: { success: true, data: product }]
|
▼
[Admin UI: product appears in list]
```

Order Placement Flow (Razorpay)

```
[Customer: CheckoutPage.jsx]
| POST /payments/create-order {addressId, couponCode, usePoints, deliveryType}
▼
[payments.controller.js: createOrder()]
|
├─ getAddressess(userId) → validate address ownership
├─ getCartByUser(userId) → load cart items
├─ pool.query(ANY($variantIds)) → batch stock pre-check
├─ validateCoupon(code, subtotal, userId) → discount
├─ findUserById(userId) → points balance → pointsDiscount cap 20%
├─ calcShipping(deliveryType, subtotal) → shipping amount
└─ createRazorpayOrder(total, 'INR', receipt) → Razorpay API
    |
    ▼
    [Razorpay API] → { id: rzp_order_id, amount, currency }
    |
    ▼
    [Response: { razorpayOrderId, amount, keyId, subtotal, discount, shipping, total }]
    |
    ▼
    [Customer: Razorpay Modal opens]
    | Customer completes payment
    ▼
    [POST /payments/verify {razorpayOrderId, paymentId, signature, ...totals}]
    |
    └─ verifySignature(orderId, paymentId, signature) → HMAC-SHA256 ✅
    └─ fulfillOrder({userId, addressId, items, totals, deliveryType...})
        |
        ▼
        [BEGIN TRANSACTION]
        └─ INSERT orders → order record
```

```

└─ INSERT order_items → line items
└─ FOR EACH variant: FOR UPDATE → validate → UPDATE stock
└─ UPDATE inventory (sync)
└─ INSERT inventory_logs
└─ UPDATE users (points)
└─ UPDATE coupons (used_count)
└─ DELETE cart_items

[COMMIT]
|
▼

[SMS: sendOrderConfirmation()] ← fire-and-forget
[Notification: createNotification()] ← fire-and-forget
|
▼

[Response: { success: true, data: { orderId } }]
|
▼

[Customer: redirect to /order-success/:orderId]

```

Stock Sync Flow

Admin sets stock via Inventory Module:

```

[PUT /inventory/variants/:variantId/stock { stock_quantity: 50 }]
|
▼

[inventory.controller.js: updateVariantStock()]
|
▼

[setVariantStock(variantId, 50, threshold, adminId, notes)]
|
└─ UPDATE product_variants SET stock = 50 WHERE id = $variantId
|
└─ INSERT INTO inventory (variant_id, stock_quantity=50)
|   ON CONFLICT (variant_id) DO UPDATE SET stock_quantity = 50
|
└─ INSERT INTO inventory_logs (action='variant_stock_set', qty=50)
|
▼

[Both product_variants.stock AND inventory.stock_quantity = 50]

```

When order is placed (inside fulfillOrder transaction):

```

[UPDATE product_variants SET stock = stock - qty WHERE id = $variantId]
[UPDATE inventory SET stock_quantity = GREATEST(stock_quantity - qty, 0) WHERE variant_id = $variantId]
[INSERT INTO inventory_logs (action='order_deduction', qty=-qty)]

```

When order is cancelled (inside cancelOrder transaction):

```

[UPDATE product_variants SET stock = stock + qty WHERE id = $variantId]
[UPDATE inventory SET stock_quantity = stock_quantity + qty WHERE variant_id = $variantId]
[INSERT INTO inventory_logs (action='return_restock', qty=+qty)]

```

EXECUTIVE SUMMARY

ShopperHub — System Maturity Assessment

One-page summary for stakeholders and engineering leadership.

What the System Does

ShopperHub is a complete fashion e-commerce platform with a React SPA frontend, Node.js/Express backend, and PostgreSQL database. It supports the full retail lifecycle: product catalogue management with hierarchical categories and variants, multi-step checkout with Razorpay online payment and COD, atomic order fulfillment with inventory synchronization, loyalty points, coupon engine, SMS notifications, and a comprehensive admin panel covering products, inventory, orders, banners, CMS pages, delivery configuration, and warehouse management.

✓ Top 3 Strengths

- → Atomic order fulfillment: fulfillOrder() uses DB transactions with FOR UPDATE row locks — prevents overselling and data corruption under concurrent orders
- → Layered architecture: strict Routes → Controllers → Services → Queries separation makes the codebase navigable and extensible
- → Security fundamentals: Helmet CSP, JWT in httpOnly cookies, bcrypt passwords, HMAC payment verification, admin role guard — solid baseline

✗ Top 3 Risks

- → Zero test coverage: no unit, integration, or E2E tests for any business logic — fulfillOrder(), verifySignature(), validateCoupon() are all untested. A regression in payment flow could go undetected.
- → Inventory gap: variant stock is NOT automatically registered in the Inventory module when a variant is added — admin must do this manually. Missed setup means warehouse tracking and low-stock alerts are silently disabled.
- → Exposed credentials in git history: Twilio SID + Fast2SMS API key were committed and need immediate rotation — anyone with repo access (including historical clones) has these credentials.

💡 Recommended Next Steps

- → Immediate (Week 1): Rotate all exposed API keys. Auto-create inventory row on variant add. Add Joi validation to checkout and auth endpoints.
- → Short-term (Month 1): Write tests for payment verification, order fulfillment, coupon validation. Add DB indexes (orders.user_id, cart_items.user_id, products.slug). Wrap bulk stock update in transaction.
- → Pre-production (Month 2): Migrate image storage to S3/Cloudflare R2. Add Redis for rate limiting and public endpoint caching. Add API versioning (/api/v1/). Set up CI/CD pipeline with test gate.

📊 Current Maturity: 5.9/10

- → Architecture: 7.5/10 — Clean, well-structured monolith
- → Security: 6.0/10 — Good foundations; missing input validation
- → Data Integrity: 7.0/10 — Transactions present; bulk update gap
- → Testing: 1.0/10 — No automated tests; critical blocker for production
- → Performance: 6.0/10 — No indexes documented; no caching layer
- → **Verdict: Strong MVP foundation. Not production-ready. Estimated 4–6 weeks of hardening required before launch.**